

EEE283

Digital Logic Design

Course Instructor: A K M Anindya Alam

Topic 10:

Sequential Circuits (Finite State Machines - FSM)

States

- A state defines the current condition of a system.
- Example:

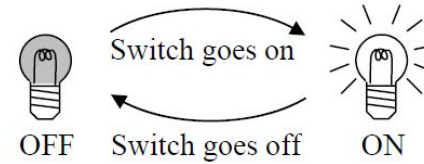


Figure 11-3 States of a Light Bulb

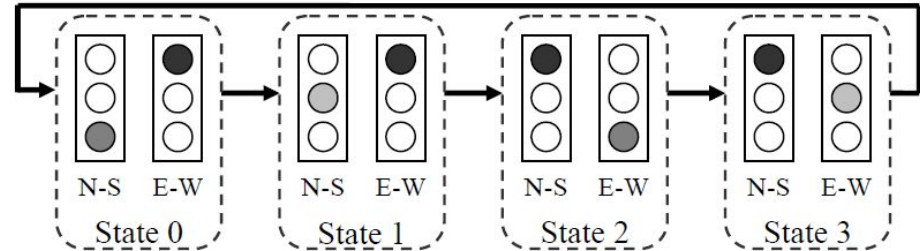
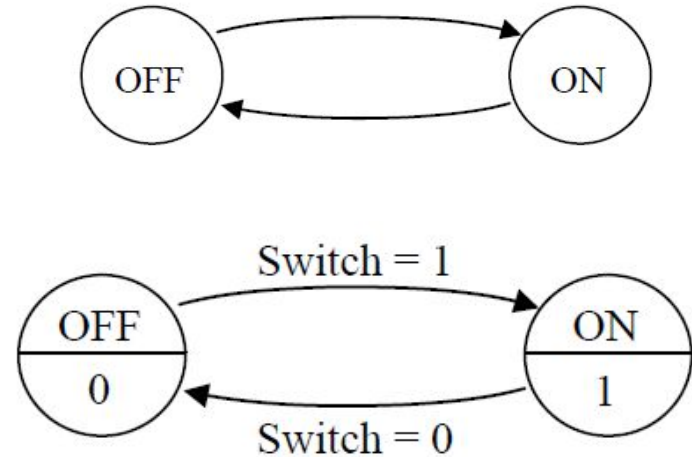
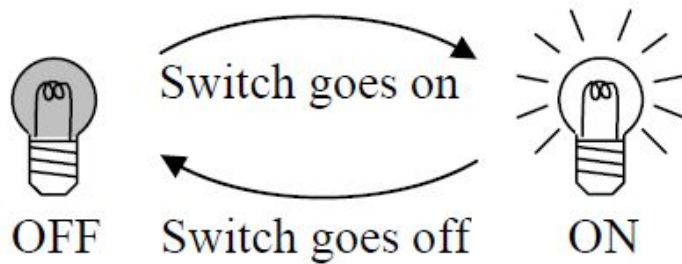


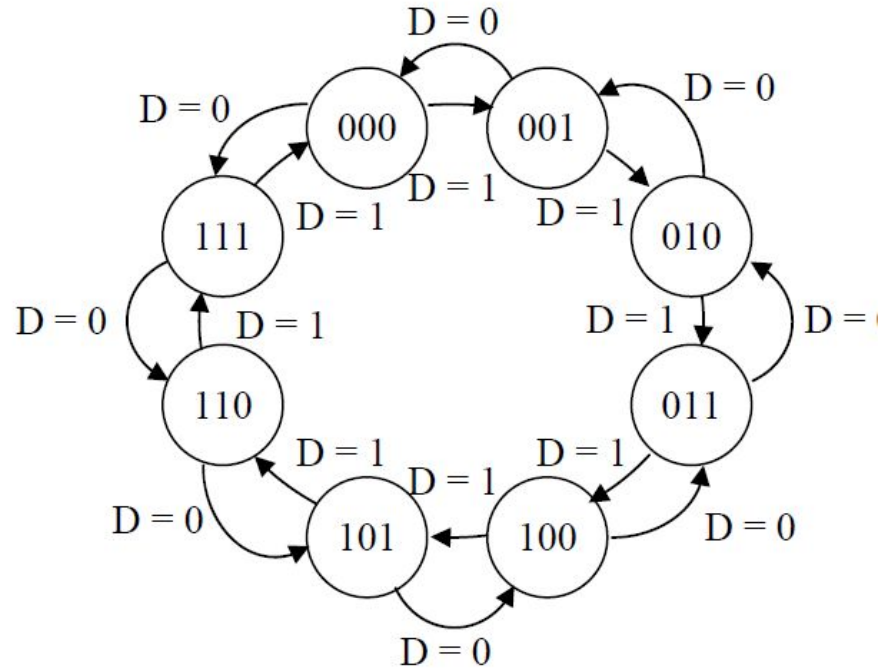
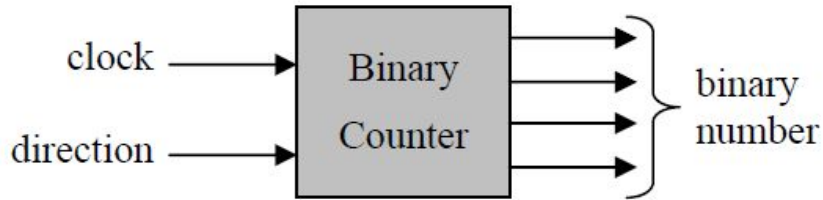
Figure 11-2 States of a Traffic Signal System

State Diagrams

- A state diagram models a state machine by using circles to represent each of the possible states and arrows to represent all of the possible transitions between the states.



State Diagram of Up/down Counter



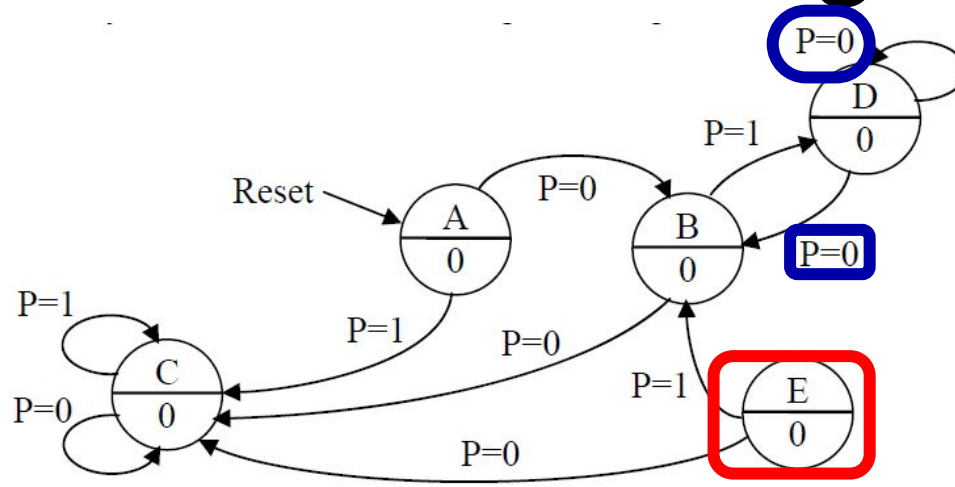
Errors in State Diagram

State diagram must fully describe the operation of a state machine.

Possible errors include the following situations.

- Any state other than an initial state that has no transitions going *into* it should be removed since it is impossible to reach that state.
- For a system with n inputs, there should be exactly 2^n transitions coming *out* of every state, one for each pattern of ones and zeros for the n inputs. Some transitions may come back to the current state, but every input must be accounted for. Missing transitions should be added while duplicates should be removed.

Errors in State Diagram



Error 1 – There is no way to get to state E. It should be removed. Although state A has no transitions to it, it is not a problem because it is the initial state.

Error 2 – The transition from state D for P=0 is defined twice while the transition for P=1 is never defined.

Block Diagram of a State Machine

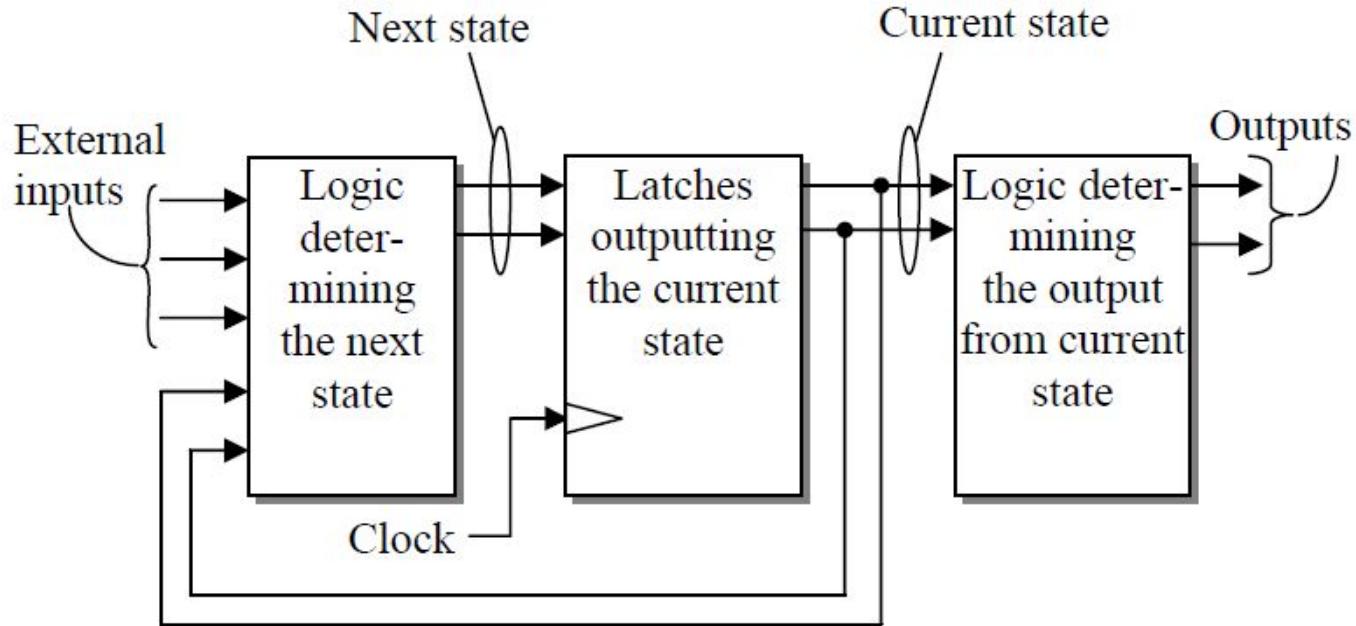


Figure 11-9 Block Diagram of a State Machine

State Machine Design: Example 1

Example 1: push button circuit used to control a light bulb.

- When the button is pressed, the light bulb turns on.
 - When the button is released, it stays on.
 - Pushing the button a second time turns off the bulb, and the bulb stays off when the button is released.
-
- Single Input: Push Button ($B=0$ or $B = 1$)
 - Single State: Light on or off ($L=0$ or $L=1$)

State Machine Design: Example 1

Table 11-1 List of States for Push Button Circuit

State	Numeric Value	
	Decimal	Binary
Bulb off; button released	0	00
Bulb on; button pressed	1	01
Bulb on; button released	2	10
Bulb off; button pressed	3	11

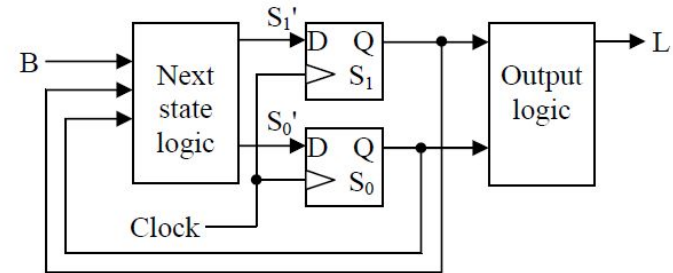
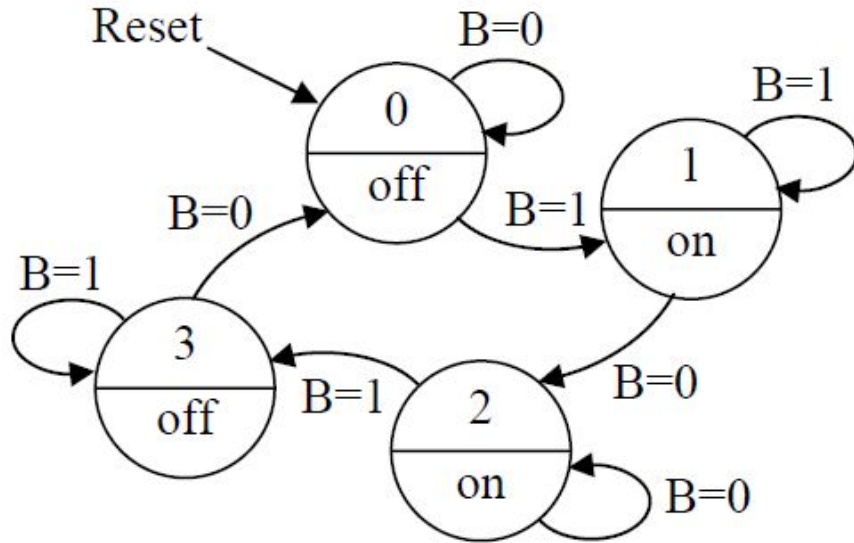


Figure 11-18 Block Diagram for Push Button Circuit

State Machine Design: Example 1

Table 11-2 Next State Truth Table for Push Button Circuit

S_1	S_0	B	S_1'	S_0'	
0	0	0	0	0	← State 0 stays in state 0 when B=0
0	0	1	0	1	← State 0 goes to state 1 when B=1
0	1	0	1	0	← State 1 goes to state 2 when B=0
0	1	1	0	1	← State 1 stays in state 1 when B=1
1	0	0	1	0	← State 2 stays in state 2 when B=0
1	0	1	1	1	← State 2 goes to state 3 when B=1
1	1	0	0	0	← State 3 goes to state 0 when B=0
1	1	1	1	1	← State 3 stays in state 3 when B=1

Table 11-3 Output Truth Table for Push Button Circuit

S_1	S_0	L	
0	0	0	← State 0: bulb is off
0	1	1	← State 1: bulb is on
1	0	1	← State 2: bulb is on
1	1	0	← State 3: bulb is off

State Machine Design: Example 1

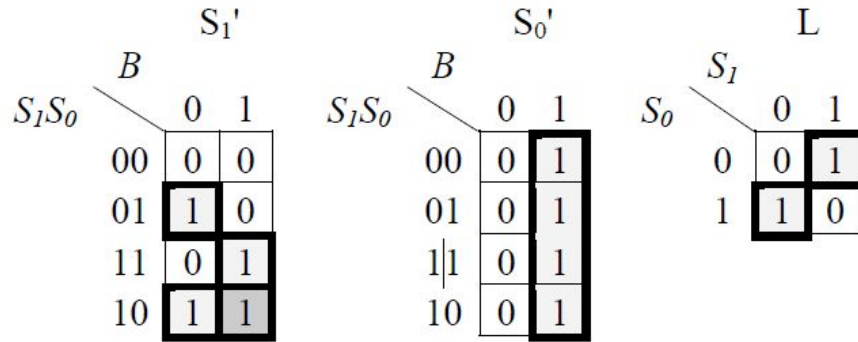


Figure 11-19 K-Maps for S_1' , S_0' , and L of Push Button Circuit

$$S_1' = \bar{S}_1 \cdot S_0 \cdot \bar{B} + S_1 \cdot B + S_1 \cdot \bar{S}_0$$

$$S_0' = B$$

$$L = \bar{S}_1 \cdot S_0 + S_1 \cdot \bar{S}_0 = S_1 \oplus S_0$$

State Machine Design: Example 1

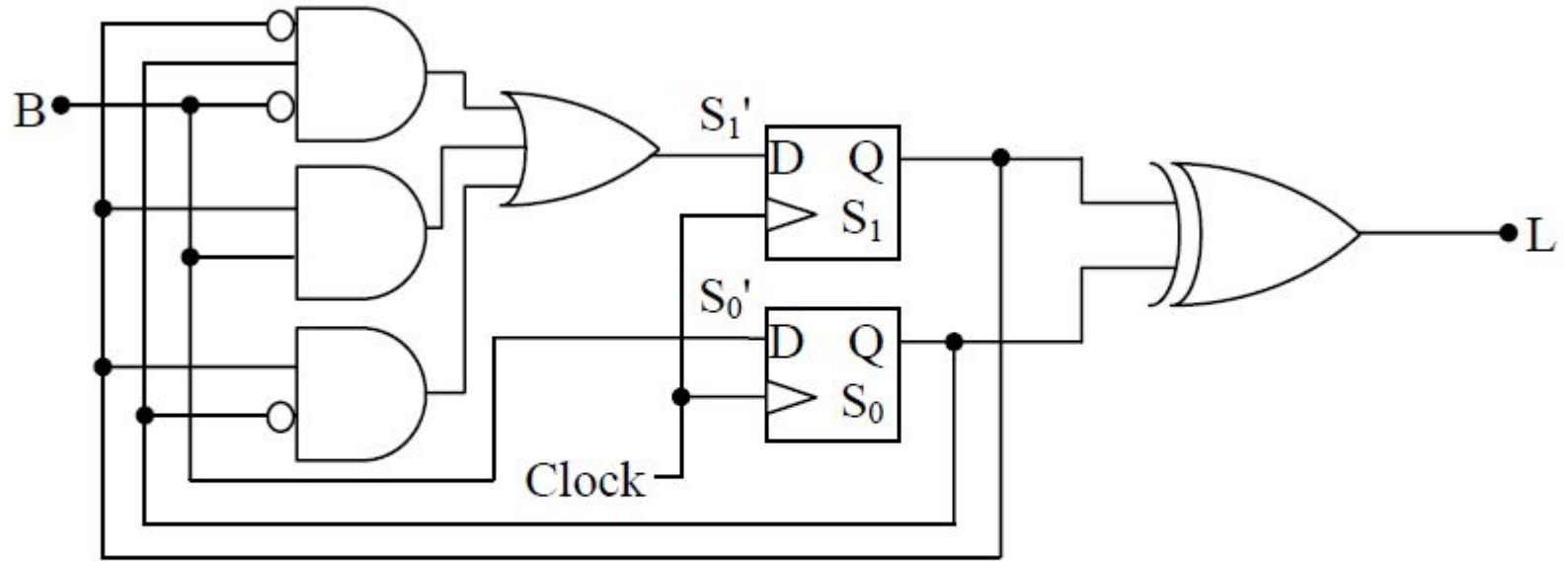


Figure 11-20 Finished Push Button Circuit

Finite State Machine

- A **finite state machine** is a sequential circuit having a limited (finite) number of states occurring in a prescribed order. A counter is an example of a state machine; the number of states is called the *modulus*.

Finite State Machine

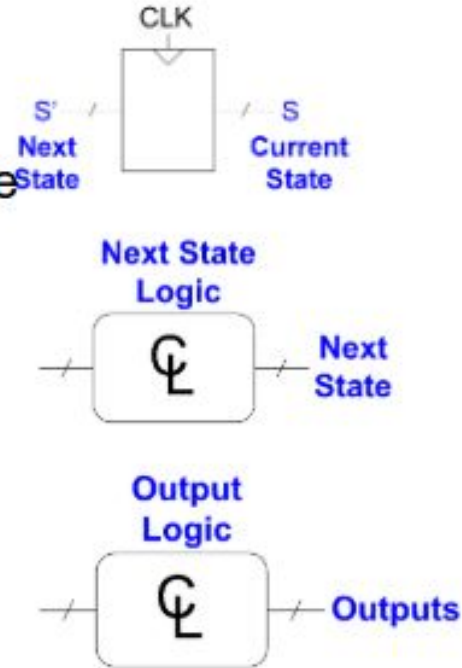
- Consists of:

- **State register**

- Stores current state
 - Loads next state at clock edge

- **Combinational logic**

- Computes the next state
 - Computes the outputs



Finite State Machine

- **Next state** determined by current state and inputs
- Two types of finite state machines differ in **output logic**:

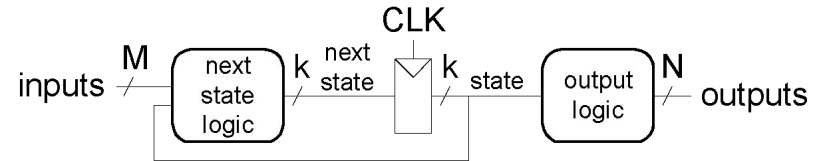
- **Moore FSM:**

outputs depend only on current state

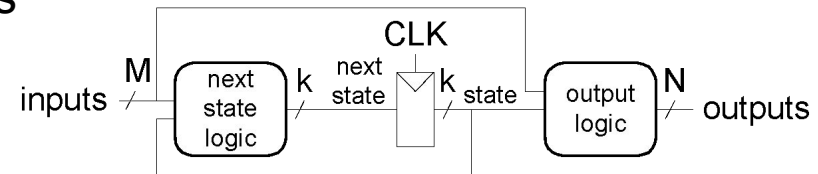
- **Mealy FSM:**

outputs depend on current state *and* inputs

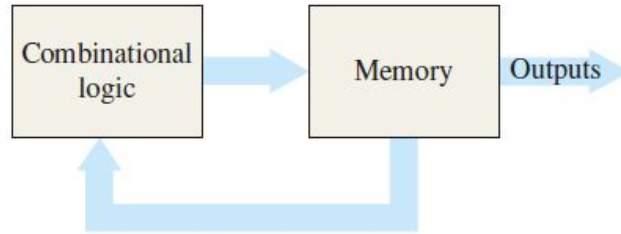
Moore FSM



Mealy FSM



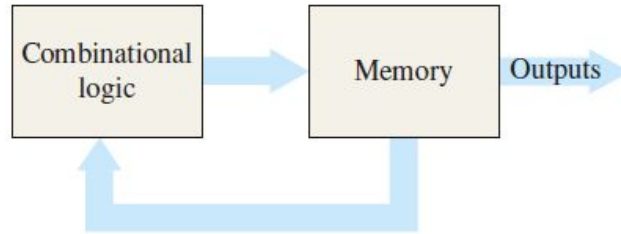
Moore Machine



(a) Moore machine

- the combinational logic is a gate array with outputs that determine the next state of the flip-flops in the memory.
- There may or may not be inputs to the combinational logic.
- If there is an input(s), it does not affect the outputs (input is dependent only on the present state of the memory).
- Output Comes from **Memory**. There may also be output combinational logic, such as a decoder.

Moore Machine



(a) Moore machine

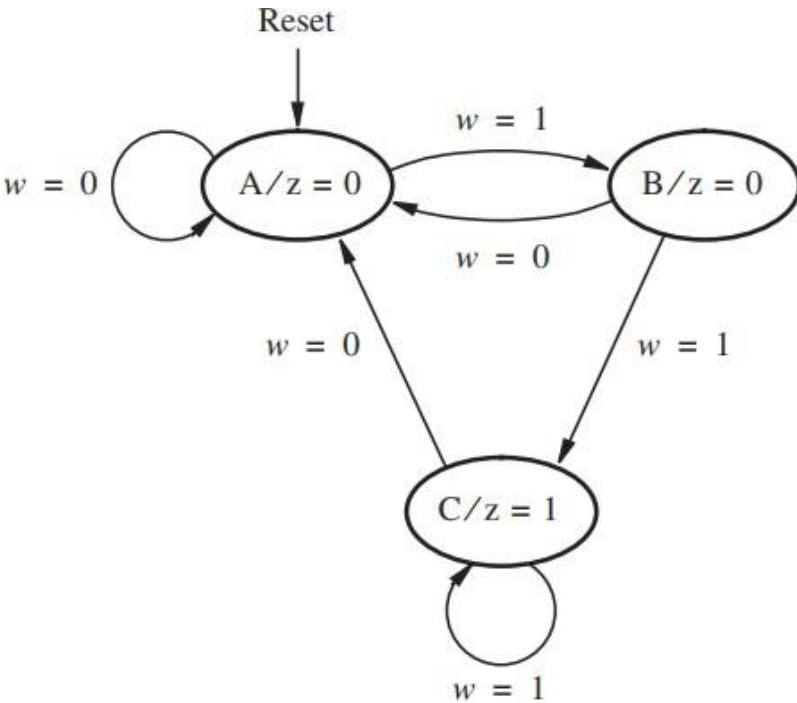
- the combinational logic is a gate array with outputs that determine the next state of the flip-flops in the memory.
- There may or may not be inputs to the combinational logic.
- If there is an input(s), it does not affect the outputs (input is dependent only on the present state of the memory).
- Output Comes from **Memory**. There may also be output combinational logic, such as a decoder.

Example of Moore Machine

- ❖ The output z is equal to 1 if during **two immediately preceding** clock cycles the input w was equal to 1. Otherwise, the value of z is equal to 0.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0

Example of Moore Machine

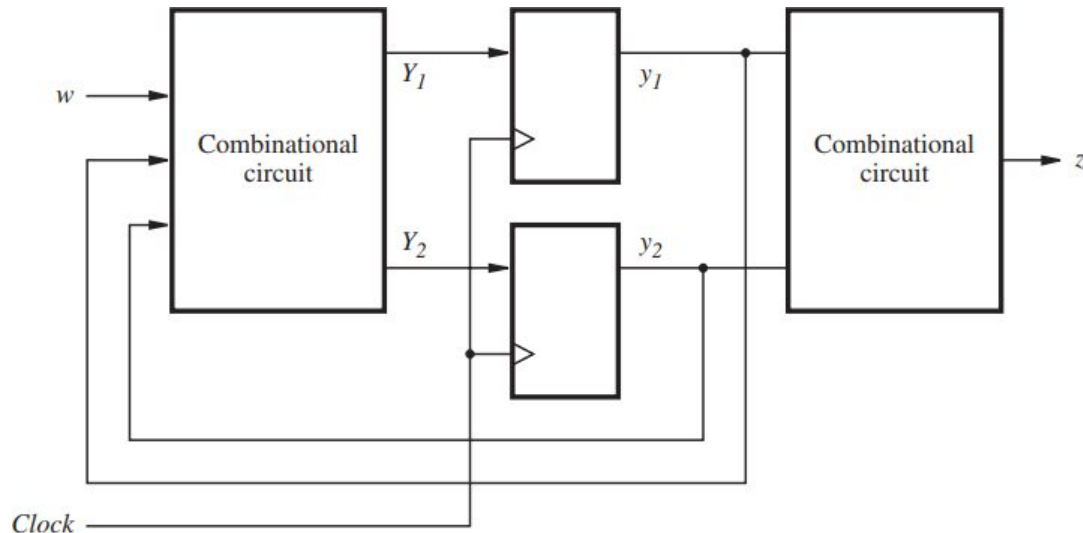


Present state	Next state		Output z
	$w=0$	$w=1$	
A	A	B	0
B	A	C	0
C	A	C	1

Example of Moore Machine

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

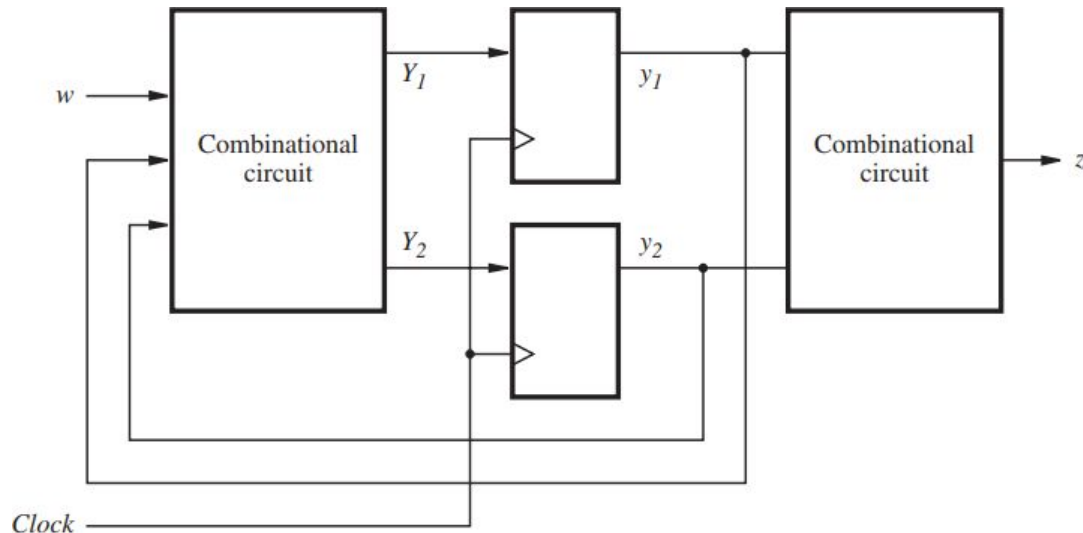
	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d



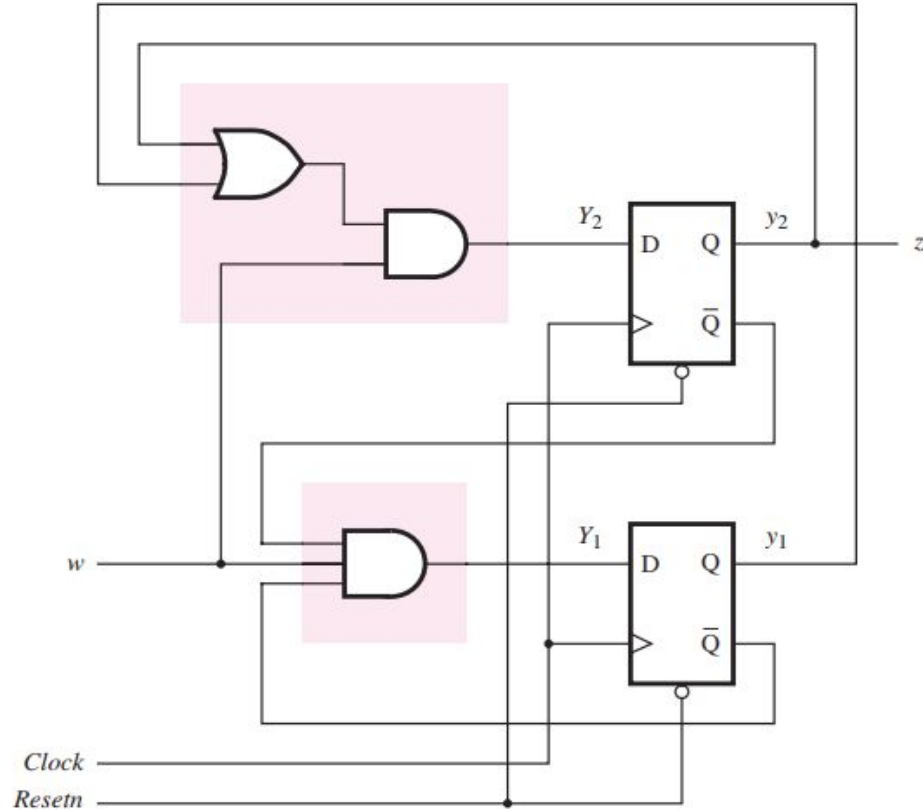
Example of Moore Machine

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

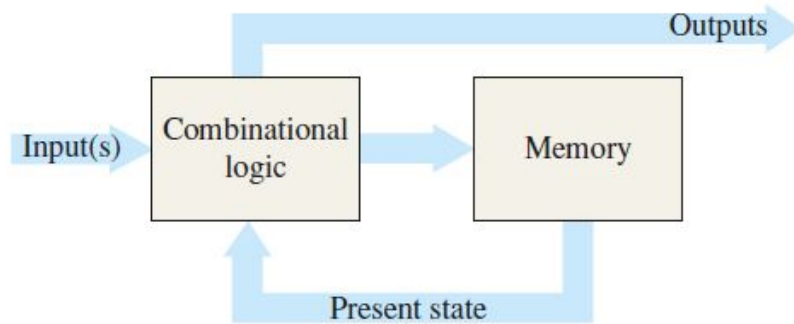
	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d



Example of Moore Machine



Mealy Machine

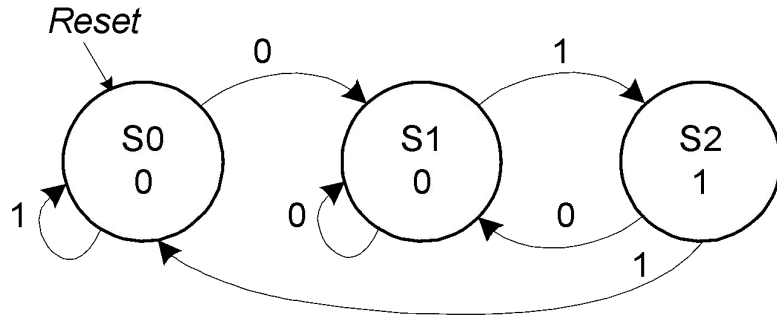


(b) Mealy machine

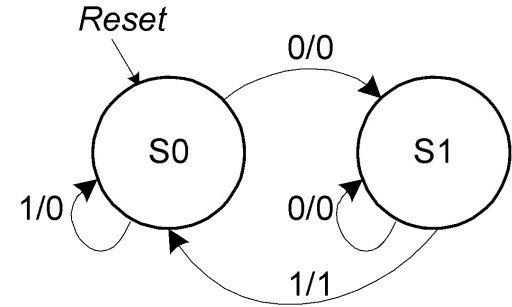
- the present state affects the outputs, just as in the Moore machine;
- but in addition, the inputs also affect the outputs.
- The outputs come directly from the **combinational logic** and not the memory.

Moore vs Mealy

Moore FSM



Mealy FSM



Moore vs Mealy

Current State		Inputs	Next State	
S_1	S_0		S'_1	S'_0
0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0

State	Encoding
S0	00
S1	01
S2	10

$$S'_1 = S_0 A$$

$$S'_0 = \overline{A}$$

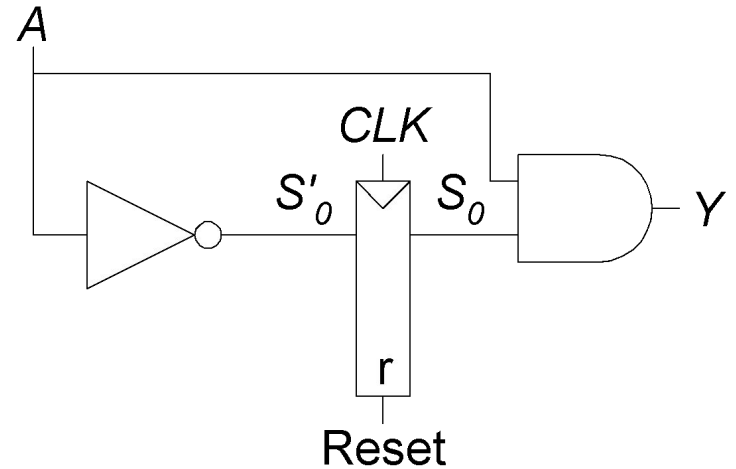
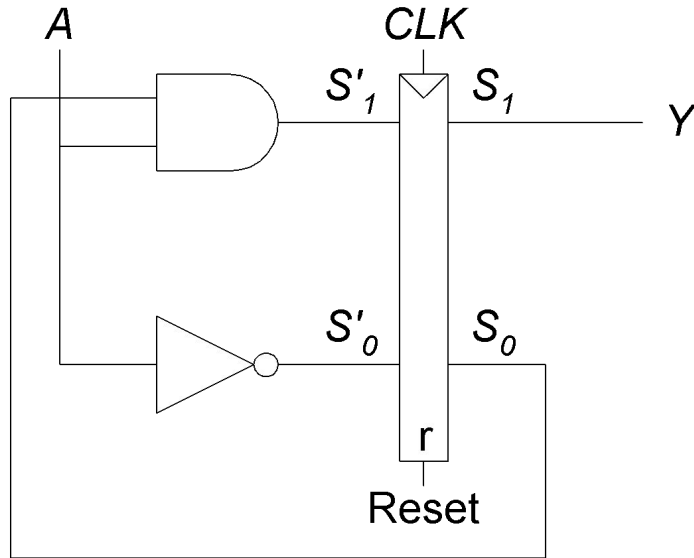
Current State	Input	Next State	Output
S_0	A	S'_0	Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

State	Encoding
S0	00
S1	01

$$S'_0 = \overline{A}$$

$$Y = S_0 A$$

Moore vs Mealy



Example: Division of a binary number by 3

Design a Finite State Machine (FSM) that will determine a binary number is divisible by 3 or not. The machine will accept a binary bit as input each clock cycle forming a binary string with each bit being added at the MSB, and will output 1 if the string is divisible by 3 and 0 if it is not.

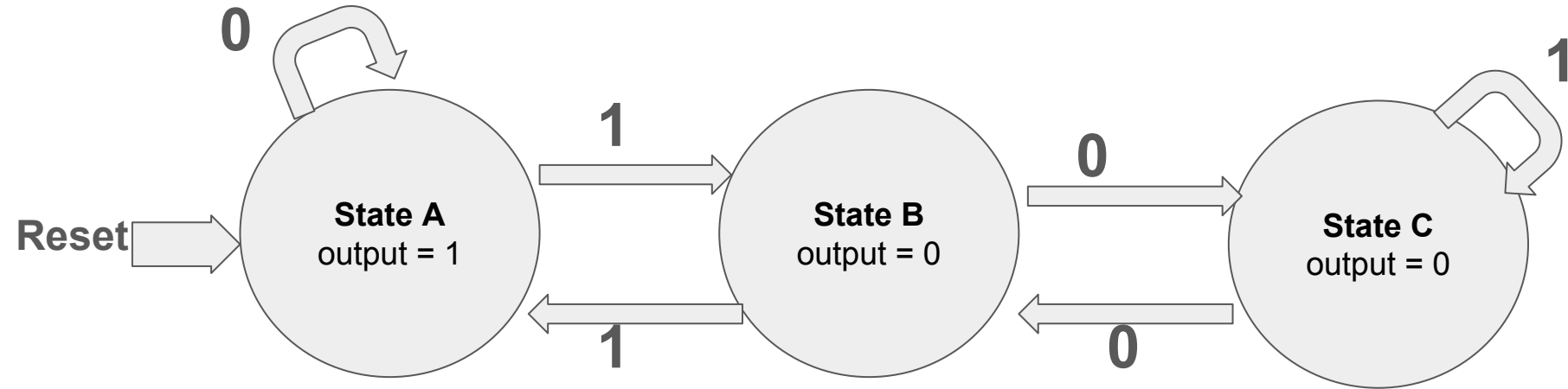
Clock Cycle	0	1	2	3	4	5
Input, B	-	1	1	0	1	1
Binary String	-	'1'	'11'	'011'	'1011'	'11011'
Output, Z	0	0	1	1	0	1

Example: Division of a binary number by 3

Design a Finite State Machine (FSM) that will determine a binary number is divisible by 3 or not. The machine will accept a binary bit as input each clock cycle forming a binary string with each bit being added at the MSB, and will output 1 if the string is divisible by 3 and 0 if it is not.

State Name	State Description	State Number	Encoded value, y_1y_0
A	Remainder = 0	0	00
B	Remainder = 1	1	01
C	Remainder = 2	2	10

Example: Division of a binary number by 3



Example: Division of a binary number by 3

Present State, y y_1y_0	Next State, Y Y_1Y_0		Output, Z
	Input, B = 0	Input, B = 1	
00	00	01	1
01	10	00	0
10	01	10	0

$$Y_0 = \overline{y_1} \overline{y_0} B + y_1 \overline{y_0} \overline{B}$$

$$Y_1 = y_1 \overline{y_0} B + \overline{y_1} y_0 \overline{B}$$

$$z = \overline{y_0} \overline{y_1}$$

Example: Division of a binary number by 3